# Phase2_Report

# Adaptive AI-Driven Intrusion Detection & Protection System using Federated Learning (A-IDAPS-FL)

## Phase 2 Project Report

**Department of Computer Science & Engineering, Madan Mohan Malaviya University of Technology, Gorakhpur**
**Project Group G-52 · Session 2025-26 · Supervisor: Dr. Raj Kumar, Assistant Professor**

Tarkeshvar Mani Yadav (2023011078) · Vishal Kumar (2023011085) · Shashwat Srivastava (2023021159) · Ritesh Pandey (2023021154)

## Abstract

Phase 1 of A-IDAPS-FL delivered a centralized hybrid intrusion detection system (Autoencoder + XGBoost) trained on the DDoS day of CIC-IDS2017 and served through a real-time Streamlit dashboard. Phase 2 turns the system into a privacy-preserving federated one. Four simulated network nodes, each seeing a different mix of attacks, train the models locally with the Flower framework and exchange only model parameters over TLS; a central server aggregates them (FedAvg for the Autoencoder, tree bagging for XGBoost), calibrates the decision thresholds on its own labelled hold-out and publishes a global model. The training data was widened to all eight CIC-IDS2017 capture days (2.83 million flows, seven attack families). On a 283,074-flow report split the federated hybrid model reaches 99.23% accuracy, 99.95% recall and 0.95% false-positive rate, against 99.55% / 99.68% / 0.49% for a centralized model that needed 2.26 million raw flow records uploaded to the server, and 88.92% / 51.75% / 1.96% for the Phase 1 model. Phase 2 also adds autonomous prevention (host-firewall blocking with audit and undo), per-prediction explainability (TreeSHAP and a LIME-style local surrogate), persistent event logging, a differential-privacy option with a formal (ε, δ) analysis, an LSTM temporal-model experiment, CSE-CIC-IDS2018 schema support and a regression test suite. During Phase 2 several defects of the Phase 1 deployment were found and corrected; they are documented in Chapter 3 because they change how the Phase 1 results must be read.

## Chapter 1 — Introduction

### 1.1 Motivation

Signature-based intrusion detection cannot see new attacks, machine-learning detection needs large labelled traffic sets, and collecting that traffic centrally is itself a privacy and compliance problem. Federated learning (McMahan et al., 2017) trains one model across many data owners without moving the data. Phase 2 applies it to the hybrid detector built in Phase 1.

### 1.2 Problem statement

Build a federated version of the A-IDAPS-FL detector in which (a) raw traffic never leaves a node, (b) the global model detects every attack family present in CIC-IDS2017, (c) transport is encrypted, (d) detection is explainable per flow, (e) confirmed attacks can be blocked automatically, and (f) the privacy, accuracy and communication trade-offs are measured, not asserted.

### 1.3 Scope of Phase 2 (10-week plan)

Federated learning study and Flower setup; TLS between nodes and server; multi-attack dataset preparation over four non-IID nodes; local Autoencoder and XGBoost training; FedAvg / bagging aggregation; differential privacy; convergence and communication-overhead measurement; centralized vs federated comparison; dashboard update; final testing and documentation.

---

## Chapter 2 — Literature Survey (additions to Phase 1)

- **Federated averaging.** McMahan et al. (2017) average client weight updates weighted by local sample count. It applies directly to the Autoencoder.
- **Federated gradient-boosted trees.** Trees have no weights to average. Flower's `FedXgbBagging` strategy (Flower Labs, 2023) lets every client grow trees on top of the shared booster and appends them; we use it for XGBoost.
- **Differential privacy in FL.** McMahan et al. (2018) clip client updates and add Gaussian noise at the server (client-level DP); the privacy budget is accounted with Rényi DP (Mironov, 2017). DP-FedAvg is designed for hundreds to thousands of clients.
- **Explainability.** SHAP (Lundberg & Lee, 2017) via TreeSHAP for the XGBoost component, and LIME (Ribeiro et al., 2016) as a model-agnostic local surrogate of the whole hybrid rule.
- **Dataset.** CIC-IDS2017 (Sharafaldin et al., 2018); CSE-CIC-IDS2018 uses the same CICFlowMeter features under abbreviated names.

---

## Chapter 3 — Audit of the Phase 1 Deployment

Before federating the Phase 1 models we replayed the CIC-IDS2017 data through the deployed code. The findings below were fixed in Phase 2 and must be kept in mind when citing Phase 1 numbers.

| Finding | Effect | Fix |
|---|---|---|
| `threshold.json` held 0.5 (the notebook's ensemble cutoff) instead of the Autoencoder reconstruction threshold 1.149e-4 | Deployed predictor had 0% attack recall on its own training day | Threshold file now carries the calibrated reconstruction threshold and the classifier cutoff |
| Live extractor used seconds and Ethernet frame lengths; the dataset uses microseconds and payload bytes; backward window never filled | Live traffic never resembled training data, which is why Phase 1 needed an XGBoost "gate" that made live detection impossible | Extractor rewritten to CICFlowMeter conventions; gate removed; unit tests |

| Finding | Effect | Fix |
|---|---|---|
| Model trained on the DDoS day only | On multi-attack data: 5% recall on port scans, 0.1% on brute force | Retrained on all eight days |
| Dashboard latency was `random.uniform(6, 20)`, confusion matrix guessed from scores, accuracy hard-coded, SHAP table static | Displayed metrics were not measurements | Measured latency, ground-truth-only confusion matrix, hold-out accuracy, per-prediction SHAP |
| The 99% "hybrid" accuracy in the Phase 1 notebook came almost entirely from XGBoost (mean DL contribution 0.018) | The Autoencoder was not the primary decision maker as documented | Decision rule redesigned (Chapter 5) |

---

## Chapter 4 — Dataset and Federated Partitioning

### 4.1 Data

All eight `MachineLearningCVE` files of CIC-IDS2017: 2,830,743 flows after cleaning (no rows lost). Labels are mapped to families: BENIGN 2,273,097; DoS 252,661 (Hulk, GoldenEye, slowloris, Slowhttptest); PortScan 158,930; DDoS 128,027; BruteForce 13,835 (FTP-Patator, SSH-Patator); WebAttack 2,180 (brute force, XSS, SQL injection); Bot 1,966; Other 47 (Infiltration, Heartbleed). The 30 features selected in Phase 1 are kept so results remain comparable. One MinMaxScaler is fitted on all rows.

### 4.2 Hold-out and its two halves

A stratified 20% (566,149 flows) is held by the server and split deterministically: even rows form the **calibration** half (thresholds), odd rows the **report** half (every number in this report). Nothing tuned on one half is measured on it.

### 4.3 Non-IID nodes

Each node models an organisation that mostly sees one kind of attack plus a random share of benign traffic.

| Node | Attack families | Rows | Attacks |
|---|---|---|---|
| 0 | DDoS, DoS | 668,245 | 304,550 |
| 1 | PortScan, Bot | 492,412 | 128,717 |
| 2 | BruteForce, WebAttack | 376,508 | 12,812 |
| 3 | almost only benign (Other) | 727,429 | 38 |

---

## Chapter 5 — Hybrid Decision Rule

```
dl_score    = clip( mse / (2 · 10 · ae_threshold), 0, 1 )        0.5 at 10× the benign 90th
percentile
```

```
ml_score    = piecewise-linear rescaling of P(attack) so that ml_score = 0.5 at the
calibrated cutoff
final_score = max(dl_score, ml_score)                              ATTACK if final_score >
0.5
```

A flow is an attack when **either** the supervised model recognises a known signature **or** the reconstruction error is far above anything benign traffic produces. Candidate rules were compared on the DDoS and PortScan days: the Phase 1 additive rule gave 89.7% accuracy at 18.8% FPR; averaging gave 97.6%; the max rule with a 10× anomaly multiplier gave 99.8% at 0.5% FPR. The multiplier and the classifier cutoff are the only tuned scalars and both are chosen on the calibration half.

---

## Chapter 6 — Federated Architecture

### 6.1 Components

| Component | Implementation |
|---|---|
| Framework | Flower 1.36, gRPC deployment mode (`start_server` / `start_client`), one process per node |
| Transport security | TLS: self-signed root CA + server certificate (`federated/gen_certs.py`); clients verify the CA |
| Autoencoder aggregation | FedAvg weighted by local benign rows; 15 rounds, 1 local epoch, batch 1024 |
| XGBoost aggregation | Bagging: each node adds 3 parallel trees per round on top of the global booster, the server appends them; 40 rounds → 480 trees; eta 0.1, depth 8, per-node `scale_pos_weight` |
| Server-side evaluation | Every round on the report half; per-round JSON logs |
| Calibration | `federated/calibrate.py`: AE threshold = benign 90th percentile; XGBoost cutoff = best F1 under 1% FPR, both on the calibration half |
| Differential privacy | Flower `DifferentialPrivacyServerSideFixedClipping` (client-level, fixed clipping + Gaussian noise), optional |
| Export | Global model, scaler, thresholds and per-family flow prototypes installed into `hybrid_model/`; Phase 1 artifacts backed up |

### 6.2 Why calibration is required under non-IID bagging

Nodes 2 and 3 hold almost no attacks, so the trees they add every round push probabilities down. The global booster ranks flows almost perfectly (AUC 0.9998 from round 1) but at a fixed 0.5 cutoff its recall is only 27% after 40 rounds. Calibrating the cutoff on the server's labelled calibration split (chosen value 0.164) restores 99.95% recall at 0.25% FPR. This is a property of federated bagging on heterogeneous data, and the calibration step is part of the design.

### 6.3 Data flow

```
node k: local flows → scale → train AE epoch / grow XGB trees → parameters or new trees
──TLS──▶ server
server: aggregate → evaluate on report half → log round → next round
final : calibrate on calib half → save global model → export to dashboard
```

---

## Chapter 7 — Results

### 7.1 Detection performance (report split: 283,074 flows, 55,741 attacks)

| Model | Detector | Accuracy | Precision | Recall | F1 | FPR |
|---|---|---|---|---|---|---|
| Phase 1 (DDoS day) | Autoencoder | 85.60% | 81.45% | 34.77% | 48.73% | 1.94% |
| Phase 1 (DDoS day) | XGBoost | 84.87% | 99.67% | 23.24% | 37.69% | 0.02% |
| Phase 1 (DDoS day) | **Hybrid** | 88.92% | 86.62% | 51.75% | 64.79% | 1.96% |
| Phase 2 centralized | Autoencoder | 86.07% | 94.76% | 30.95% | 46.66% | 0.42% |
| Phase 2 centralized | XGBoost | 99.88% | 99.72% | 99.68% | 99.70% | 0.07% |
| Phase 2 centralized | **Hybrid** | 99.55% | 98.04% | 99.68% | 98.86% | 0.49% |
| Phase 2 federated | Autoencoder | 85.33% | 91.00% | 28.28% | 43.15% | 0.69% |
| Phase 2 federated | XGBoost | 99.77% | 98.92% | 99.94% | 99.43% | 0.27% |
| Phase 2 federated | **Hybrid** | **99.23%** | 96.27% | **99.95%** | 98.07% | 0.95% |

Recall per attack family, hybrid rule:

| Family | Phase 1 | Centralized | Federated |
|---|---|---|---|
| DDoS | 99.99% | 99.97% | 99.97% |
| DoS | 60.49% | 99.93% | 99.92% |
| PortScan | 5.27% | 100% | 100% |
| BruteForce | 0.14% | 99.93% | 100% |
| Bot | 5.94% | 66.83% | 97.52% |
| WebAttack | 3.72% | 60.00% | 99.53% |

The federated model loses 0.3 points of accuracy to the centralized one (a higher false-positive rate from the autoencoder component) but detects the two rare families better because per-node `scale_pos_weight` and the calibrated cutoff give minority attacks more weight than one global weight does.

### 7.2 Convergence

- Autoencoder: benign reconstruction error falls from 2.4e-1 to 2.8e-5 over 15 rounds; recall reaches 99% of its final value at round 4.

- XGBoost: AUC 0.9998 at round 1 and flat thereafter; logloss keeps falling to 0.171 at round 40.

### 7.3 Communication overhead

| Quantity | Value |
|---|---|
| Centralized: raw rows that must be uploaded | 2,264,594 flows = 274 MB |
| Federated Autoencoder | 117 KB per node per round; 933 KB per round; 14.0 MB for 15 rounds |
| Federated XGBoost | 256.6 MB for 40 rounds (the growing 3.1 MB global booster is redistributed to 4 nodes every round) |
| Federated total | 270.6 MB, i.e. 0.99 × the raw upload, but consisting only of parameters and trees |

The XGBoost traffic is dominated by redistributing the full booster; sending only the trees added since the previous round would cut it to roughly 12 MB. This optimisation is noted as future work.

### 7.4 Differential privacy

Client-level $(\varepsilon, \delta)$-DP with $\delta = 1e-5$, Rényi accounting of the Gaussian mechanism, 4 clients participating every round (noise std per weight = $z \cdot C / 4$):

| Setting | ε | Benign MSE | AE recall | Usable |
|---|---|---|---|---|
| no DP | ∞ | 2.8e-5 | 0.282 | reference |
| z = 0.3, C = 1.0, 15 rounds | 145 | 1.5e-1 | 0 | no |
| z = 0.05, C = 20, 15 rounds | 3,372 | 1.9e-1 | 0 | no |
| z = 0.01, C = 1.0, 15 rounds | 76,901 | 9.6e-4 | 0.005 | no |
| z = 0.01, C = 1.0, 40 rounds | 203,151 | 3.3e-5 | 0.300 | accuracy yes, privacy no |

**Conclusion.** With four clients the noise required for a meaningful ε (below 10) is larger than the model weights themselves, and the noise level at which the Autoencoder still trains gives an ε that offers no privacy. DP-FedAvg needs the noise to average over hundreds of clients. In this deployment privacy therefore rests on the federated architecture itself (raw traffic never leaves a node) and on TLS, not on differential privacy. The pipeline supports DP end to end so the experiment can be repeated with more simulated nodes.

### 7.5 Temporal model (LSTM) experiment

Windows of 10 consecutive flows (chronological order within each capture day), chronological 70/30 split per day, window labelled ATTACK if any flow is an attack; LSTM(64) → Dense(32) → sigmoid, 4 epochs, class-weighted.

| Detector (window level) | Accuracy | Precision | Recall | F1 | FPR |
|---|---|---|---|---|---|
| LSTM over 10-flow windows | 88.91% | 78.96% | 62.69% | 69.89% | 4.31% |
| Federated hybrid, any flow in window | 95.93% | 83.92% | 99.16% | 90.90% | 4.91% |

The LSTM matches the hybrid on DDoS (99.4%) and PortScan (96.9%) but fails on DoS (0.3%), BruteForce (1.4%) and Bot (23.9%): under the chronological split those families appear as different sub-attacks in the training and test portions of a day (for example FTP-Patator in the morning, SSH-Patator in the afternoon), so the sequence model is asked to generalise to variants it never saw, which the flow-level model handles through its shared feature representation. A sequence model is therefore not adopted for Phase 2 production; the experiment is kept as a documented negative result with a reproducible script.

### 7.6 Live sniffer

The corrected extractor passes six unit tests on synthetic packets (units, direction, payload lengths, UDP window convention, bounded cache). Single-flow inference latency measured in the dashboard is 30–40 ms cold and about 2 ms warm. Capture on a real interface requires Npcap and Administrator rights and was not part of the automated tests.

---

## Chapter 8 — Protection: Autonomous Prevention

`core/prevention.py` implements the "Protection" half of the project name.

- **Policy.** A source is blocked only when its flows scored ≥ 0.75 at least 3 times within 60 s. Loopback, link-local, multicast, private (LAN) addresses, the host's own addresses, the simulator and an allowlist are never blocked.
- **Backends.** Windows `netsh advfirewall` rule per IP; Linux `iptables` DROP rule; **dry-run** (decide and log only) is the default and is what the dashboard toggle enables first.
- **Audit and undo.** Every skip, block and unblock is appended to `logs/prevention.jsonl` and to the SQLite event store; blocked IPs persist across restarts and can be unblocked from the dashboard.
- **Tests.** The policy is unit-tested in dry-run mode (thresholds, hit counting, exclusions, undo).

---

## Chapter 9 — Explainability and Persistence

- **TreeSHAP** (exact, via XGBoost `pred_contribs`) shows which features pushed the classifier toward ATTACK for the specific flow.
- **Autoencoder error share** shows which features the anomaly detector could not reconstruct.
- **LIME-style local surrogate** perturbs the flow 500 times, weights samples by proximity and fits a ridge regression to the *final hybrid score*, giving a model-agnostic view of the whole decision, with the local fit $R^2$ reported.
- **Event store.** `logs/ids_events.sqlite` keeps every verdict, alert and prevention action, closing the Phase 1 "no persistent logging" limitation.

---

## Chapter 10 — Dashboard (Phase 2)

Measured latency; ground-truth-only confusion matrix (simulated flows carry their true label, live flows are counted separately); hold-out accuracy from `model_info.json`; attack simulator seeded from the median real flow of each family with optional knob overrides; per-flow XAI panel with LIME toggle; autonomous-prevention panel (enable, apply-to-firewall, blocked list with unblock); federated-training panel (node partitions, per-round recall/FPR curves, traffic, TLS/DP flags, DP and LSTM tables, centralized-vs-federated table). Verified with Streamlit's `AppTest` harness end to end.

---

## Chapter 11 — Testing

| Suite | Tests | Covers |
|---|---|---|
| `tests/test_features.py` | 6 | extractor units, direction, payload lengths, UDP, cache bound |
| `tests/test_predictor_regression.py` | 5 | pure decision rule, threshold-file sanity, replay of 20,000 real flows (recall ≥ 0.90, FPR ≤ 0.05, per-family floors), single vs batch API |
| `tests/test_phase2_modules.py` | 5 | prevention policy (dry-run), DP accounting monotonicity, event store, LIME shape |

All 16 tests pass against the exported federated model. The replay regression is the guard that would have caught the Phase 1 threshold defect.

---

## Chapter 12 — Conclusion and Future Work

Phase 2 delivers a federated, TLS-secured, explainable and self-protecting IDS whose global model detects all CIC-IDS2017 attack families at 99.23% accuracy and 99.95% recall without any raw flow leaving a node, at a communication cost comparable to a one-time raw upload. Two negative results are reported honestly: differential privacy is not meaningful with four clients, and a 10-flow LSTM does not beat the flow-level hybrid under a strict chronological split.

Future work: (1) send only per-round tree deltas to cut XGBoost traffic by ~20×; (2) repeat the DP study with 50–200 simulated clients; (3) evaluate on CSE-CIC-IDS2018 with the schema mapping already in `prepare_data.py`; (4) federate the LSTM and test on longer, source-grouped sequences; (5) client authentication keys in addition to server TLS; (6) live validation against controlled attack traffic in an isolated lab network.

---

## References


1. McMahan, B. et al. *Communication-efficient learning of deep networks from decentralized data.* AISTATS 2017.
2. McMahan, B. et al. *Learning differentially private recurrent language models.* ICLR 2018.
3. Mironov, I. *Rényi differential privacy.* IEEE CSF 2017.
4. Beutel, D. J. et al. *Flower: A friendly federated learning research framework.* 2020.
5. Buczak, A. L., Guven, E. *A survey of data mining and machine learning methods for cyber security intrusion detection.* IEEE Comm. Surveys & Tutorials 2015.

6. Garcia-Teodoro, P. et al. *Anomaly-based network intrusion detection: techniques, systems and challenges.* Computers & Security 2009.
7. Lundberg, S. M., Lee, S.-I. *A unified approach to interpreting model predictions.* NeurIPS 2017.
8. Ribeiro, M. T., Singh, S., Guestrin, C. *"Why should I trust you?" Explaining the predictions of any classifier.* KDD 2016.
9. Sharafaldin, I., Lashkari, A. H., Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization.* ICISSP 2018.
10. Chen, T., Guestrin, C. *XGBoost: A scalable tree boosting system.* KDD 2016.